code

```
#  Smart Travel Planning using AO* Algorithm + ML Pipeline
#  Problem Statement: AI in Daily Life – Bengaluru to Chennai

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LinearRegression
from sklearn.metrics import accuracy_score, r2_score, mean_squared_error
from sklearn.preprocessing import LabelEncoder

# ── Dataset
data = {
   'Route':    ['BLR-HSR','BLR-KRG','HSR-VLR','KRG-AMB','KRG-VLR',
                'AMB-VLR','VLR-MAS','BLR-MAS','HSR-MAS','KRG-MAS'],
   'Distance': [40, 90, 110, 60, 70, 35, 130, 350, 290, 220],
   'Cost':     [120,250,300,180,200,100,350,950,800,600],
   'Time':     [0.8,2.0,2.5,1.2,1.5,0.7,2.8,6.5,5.5,4.0],
   'Class':    ['Budget','Standard','Standard','Budget','Standard',
                'Budget','Premium','Premium','Premium','Standard']
}
df = pd.DataFrame(data)
print("Dataset:\n", df.to_string(index=False), "\n")

# ================================================================
```

```
#  PART 1 — AO* Algorithm
print("="*55)
print("PART 1: AO* Algorithm — Smart Travel Planning")
print("="*55)

# AND-OR graph: {node: [(arc_cost, [children], node_type)]}
# node_type: 'OR' = pick best child | 'AND' = need ALL children
GRAPH = {
   'Bengaluru':   [(40,  ['Hosur'],       'OR'),
                   (90,  ['Krishnagiri'], 'OR')],
   'Hosur':       [(160, ['Krishnagiri', 'Vellore'], 'AND')],
   'Krishnagiri': [(60,  ['Ambur'],   'OR'),
                   (70,  ['Vellore'], 'OR')],
   'Ambur':       [(35,  ['Vellore'],  'OR')],
   'Vellore':     [(130, ['Chennai'],  'OR')],
   'Chennai':     []
}
HEURISTIC = {
   'Bengaluru':10, 'Hosur':7, 'Krishnagiri':5,
   'Ambur':3,     'Vellore':2, 'Chennai':0
}

def ao_star(node, costs, best, visited=None):
   if visited is None:
      visited = set()
   if node in visited:
      return costs[node]
   visited.add(node)
```

```
    if node == 'Chennai':
        costs[node] = 0
        return 0

    min_cost = float('inf')
    best_children = None

    for arc_cost, children, ntype in GRAPH[node]:
        for child in children:
            ao_star(child, costs, best, visited)
        if ntype == 'AND':
            total = arc_cost + sum(costs[c] for c in children)
        else:
            total = arc_cost + min(costs[c] for c in children)
        if total < min_cost:
            min_cost = total
            best_children = children

    costs[node] = min_cost
    best[node]  = best_children
    return min_cost

costs = {n: HEURISTIC[n] for n in HEURISTIC}
best  = {}
ao_star('Bengaluru', costs, best)

# Extract optimal path
path, node = [], 'Bengaluru'
while node != 'Chennai':
```

```
    path.append(node)
    node = best[node][0]
path.append('Chennai')

print(f"Optimal Path : {' -> '.join(path)}")
print(f"Optimal Cost : {costs['Bengaluru']}")
print()
print("Node f-values:")
for n, c in costs.items():
    print(f"  {n:15s}: {c}")

#  PART 2 — Data Preprocessing
print("\n" + "="*55)
print("PART 2: Data Preprocessing (3 Scaling Techniques)")
print("="*55)

features = df[['Distance', 'Cost', 'Time']].values

scalers = {
    'Min-Max Normalisation': MinMaxScaler(),
    'Z-Score Standardisation': StandardScaler(),
    'Robust Scaling': RobustScaler()
}
scaled_data = {}
for name, scaler in scalers.items():
    scaled = scaler.fit_transform(features)
    scaled_data[name] = scaled
    df_s = pd.DataFrame(scaled, columns=['Distance','Cost','Time'])
    print(f"\n{name}:")
```

```
    print(df_s.round(4).to_string(index=False))

# Plot comparison
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
for ax, (name, arr) in zip(axes, scaled_data.items()):
    ax.boxplot(arr, labels=['Dist','Cost','Time'], patch_artist=True,
               boxprops=dict(facecolor='#D6E4F0'))
    ax.set_title(name, fontsize=10, fontweight='bold')
    ax.set_ylabel('Scaled Value')
    ax.grid(True, alpha=0.3)
plt.suptitle('Scaling Comparison — Travel Dataset', fontsize=12)
plt.tight_layout()
plt.savefig('scaling_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

#  PART 3 — KNN Classification

print("\n" + "="*55)
print("PART 3: KNN Classification (k=3)")
print("="*55)

X = MinMaxScaler().fit_transform(features)  # Min-Max for KNN
le = LabelEncoder()
y = le.fit_transform(df['Class'])          # Budget=0,Premium=1,Standard=2

# Train on first 7, test on last 3
X_train, X_test = X[:7], X[7:]
y_train, y_test = y[:7], y[7:]
```

```
knn = KNeighborsClassifier(n_neighbors=3, metric='euclidean')
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)

print(f"Actual    : {le.inverse_transform(y_test).tolist()}")
print(f"Predicted : {le.inverse_transform(y_pred).tolist()}")
print(f"Accuracy  : {accuracy_score(y_test, y_pred) * 100:.1f}%")

# Predict a new query: Distance=80, Cost=220, Time=1.5
new_query = np.array([[80, 220, 1.5]])
new_scaled = MinMaxScaler().fit(features).transform(new_query)
pred_class = le.inverse_transform(knn.predict(new_scaled))[0]
print(f"New Query  (80 km, ₹220, 1.5 hrs) → Class: {pred_class}")

#  PART 4 — Linear Regression

print("\n" + "="*55)
print("PART 4: Linear Regression — Cost Prediction")
print("="*55)

X_reg = df[['Distance']].values
y_reg = df['Cost'].values

reg = LinearRegression()
reg.fit(X_reg, y_reg)
y_pred_r = reg.predict(X_reg)

print(f"Intercept  β₀ = {reg.intercept_:.2f}")
print(f"Coefficient β₁ = {reg.coef_[0]:.2f}")
```

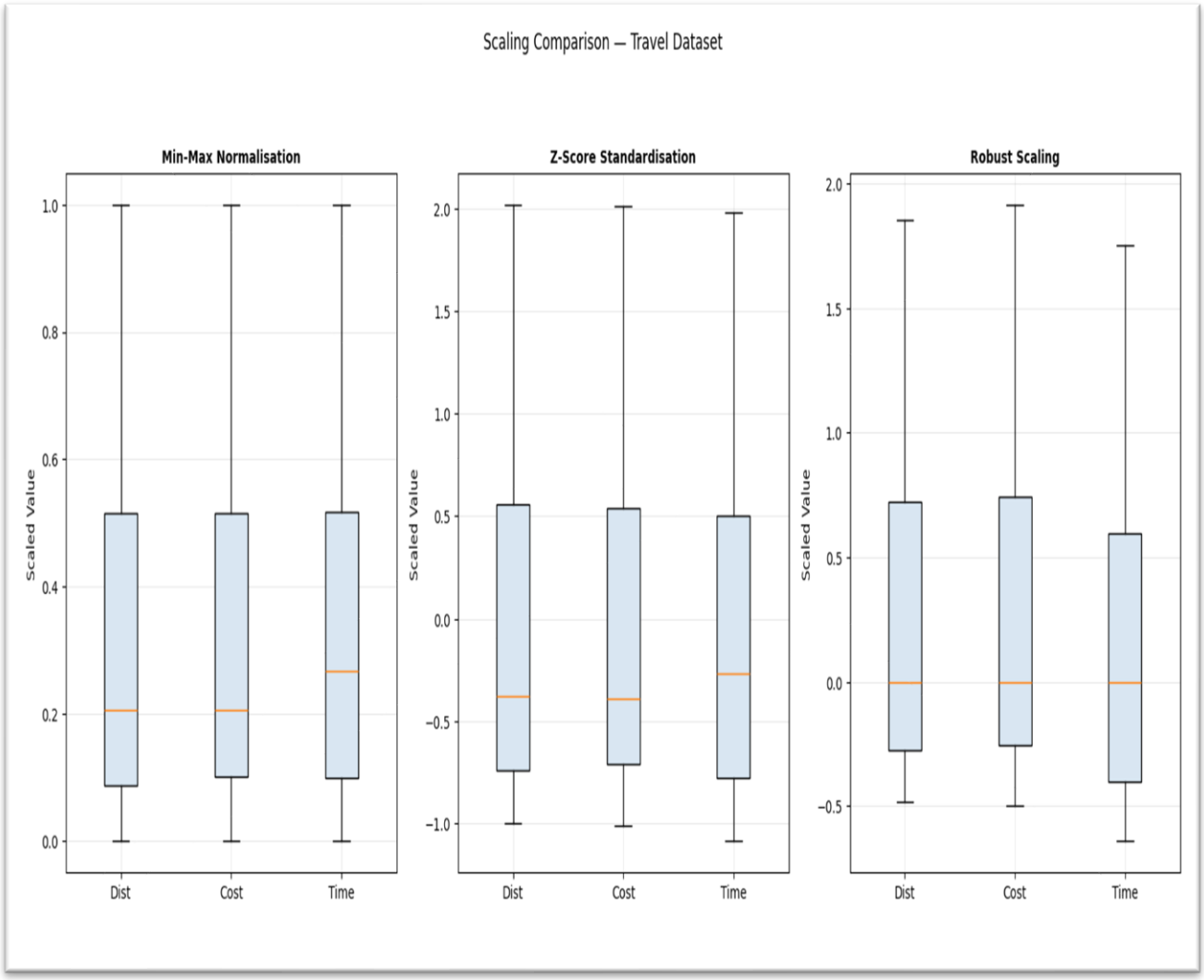
```
print(f"Equation   : Cost = {reg.intercept_:.2f} + {reg.coef_[0]:.2f} × Distance")
print(f"R² Score   : {r2_score(y_reg, y_pred_r):.4f}")
print(f"RMSE       : {mean_squared_error(y_reg, y_pred_r, squared=False):.2f}")

new_dist = np.array([[80]])
pred_cost = reg.predict(new_dist)[0]
print(f"Predicted cost for 80 km : ₹{pred_cost:.2f}")

# Plot regression
plt.figure(figsize=(8, 5))
plt.scatter(X_reg, y_reg, color='steelblue', s=80, label='Actual Data')
plt.plot(X_reg, y_pred_r, color='orangered', lw=2, label='Regression Line')
plt.scatter([80], [pred_cost], color='green', s=130, zorder=5,
            label=f'Query: 80 km → ₹{pred_cost:.0f}')
plt.xlabel('Distance (km)')
plt.ylabel('Cost (₹)')
plt.title('Linear Regression — Travel Cost Prediction')
plt.legend()
plt.grid(True, alpha=0.3)
plt.savefig('regression_plot.png', dpi=150, bbox_inches='tight')
plt.show()

print("\n" + "="*55)
print("All 4 parts complete. Graphs saved.")
print("="*55)
```

Output :


Scaling Comparison — Travel Dataset
Min-Max Normalisation
Z-Score Standardisation
Robust Scaling
Scaled Value
Dist
Cost
Time